WINTER EXAMINATION SOLUTIONS

B.Tech VI Semester • Computer Engineering

Subject Code:	BTCOC601_Y23	**Subject Name:**	Compiler Design
Created By:	@prantehare	**Total Duration:**	3 Hours

Quick Revision Note: *This solution packet maps out complete descriptive explanations and diagrams for every question. Question 1 is compulsory (12 marks). Answer any 4 questions from Questions 2 to 6. All answers are designed with high precision to help students achieve maximum scores.*

QUESTION NO. 1 (Compulsory Objective Pack) [12 Marks]

No.	Question Statement & Options	Correct Answer & Quick Context
1	**Which of the following is NOT a phase of a compiler?** a. Lexical analysis b. Semantic analysis c. Memory management d. Code optimization	**c. Memory management** Memory management is managed during runtime by the operating system or language execution environment, not inside a compiler translation pipeline.
2	**In which phase of the compiler is the syntax tree generated?** a. Lexical analysis b. Syntax analysis	**b. Syntax analysis** The syntax analyzer (parser) reads the tokens sent from the lexical analyzer and groups them into a parse tree or abstract syntax tree.

No.	Question Statement & Options	Correct Answer & Quick Context
	c. Semantic analysis d. Code generation	
3	**Which data structure is typically used to represent the syntax tree during the parsing phase?** a. Linked list b. Stack c. Tree d. Array	**c. Tree** Because programming structures (loops, expressions, conditions) are hierarchical and nested, a non-linear tree layout is naturally used.
4	**Which phase of the compiler is responsible for detecting semantic errors in the program?** a. Lexical analysis b. Syntax analysis c. Semantic analysis d. Code generation	**c. Semantic analysis** This phase verifies context requirements like type compatibility, tracking declarations, and resolving scope rules.
5	**The final phase of a compiler that generates the machine code is:** a. Code generation b. Code optimization c. Lexical analysis d.	**a. Code generation** The target code generator converts optimized intermediate symbols into native assembly or binary machine language instructions.

No.	Question Statement & Options	Correct Answer & Quick Context
	Semantic analysis	
6	**Which of the following is NOT a type of intermediate code representation?** a. Three-address code b. Abstract syntax tree c. Assembly code d. Bytecode	**c. Assembly code** Assembly code is a low-level target machine language specific to hardware. Intermediate representations are strictly architecture-independent.
7	**The output of the lexical analysis phase of a compiler is:** a. A syntax tree b. Tokens c. Machine code d. Intermediate code	**b. Tokens** The lexical phase breaks down a continuous sequence of raw code characters into clean, labeled chunks called tokens.
8	**In the context of compiler construction, LR parsing refers to:** a. A type of lexical analyzer b. A method of parsing using a left-to-right scan and a rightmost derivation in	**b. A method of parsing using a left-to-right scan and a rightmost derivation in reverse** **L** means left-to-right reading; **R** means constructing the reverse order of a rightmost derivation step via reductions.

No.	Question Statement & Options	Correct Answer & Quick Context
	reverse c. A type of intermediate code d. A method for code optimization	
9	**The semantic analysis phase of the compiler mainly checks for:** a. Syntax errors b. Type mismatches and scope errors c. Efficiency of the program d. Token generation	**b. Type mismatches and scope errors** It ensures operators are used with correct data types and that variables have been declared within their proper scope.
10	**Which of the following is a common type of grammar used in compiler construction?** a. Context-free grammar b. Regular grammar c. Context-sensitive grammar d. All of the above	**d. All of the above** Regular grammar specifies lexemes, context-free grammar maps program syntax structures, and context conditions handle static restrictions.

No.	Question Statement & Options	Correct Answer & Quick Context
11	**A compiler's code optimization phase aims to:** a. Detect syntax errors b. Minimize the runtime/resource utilization of the program c. Translate high-level code to assembly d. Create an abstract syntax tree	**b. Minimize the runtime/resource utilization of the program** It alters instructions so they run faster and use less memory while preserving the exact original logic.
12	**The main task of a lexical analyzer is to:** a. Parse the source code b. Translate into machine code c. Break the source code into tokens d. Optimize the generated code	**c. Break the source code into tokens** It filters out spaces or comments and groups character matches together into recognized token streams.

QUESTION NO. 2 (Syntax Analysis Core) [12 Marks]

A) What are the differences between top-down and bottom-up parsing? [6 Marks]

Top-down and bottom-up parsers represent two opposite strategies for building a syntax tree from input tokens.

Property	Top-Down Parsing	Bottom-Up Parsing
Direction of Growth	Starts from the high-level Root symbol and builds downward to match the token leaves.	Starts from the low-level input tokens (leaves) and reduces them upward to meet at the Root.
Derivation Method	Traces a **Leftmost Derivation**.	Traces a **Rightmost Derivation in Reverse**.
Basic Operations	Tokens are matched and structural rules are expanded.	Uses **Shift** (pushing onto a stack) and **Reduce** (replacing strings with a non-terminal).
Grammar Limitations	Cannot handle left-recursive rules directly; requires converting the grammar first.	Can handle left-recursive rules directly without any adjustments. Highly powerful.
Key Frameworks	Recursive Descent, LL(1) Table-Driven parsing.	SLR(1), LALR(1), Canonical LR(1) parsing models.

B) Explain the working of LL(1) and LR(1) parsers? [6 Marks]

1. Mechanics of LL(1) Parsers:

An LL(1) parser reads input from **L**eft-to-right, creates a **L**eftmost derivation, and uses exactly **1** lookahead token to make decisions. The system manages state using a stack, an input buffer, and a hard-coded lookup matrix table.

During a calculation step, it checks the top symbol of the stack (X) against the current input character (a):

- If X matches a and both are equal to the end-marker $\$$, parsing completes successfully.
- If X matches a (terminals), the item is popped from the stack and the input advances.
- If X is a non-terminal symbol, the parser checks table cell $M[X, a]$. If the cell contains a production rule like $X \rightarrow Y_1 Y_2$, it pops X and pushes the replacement symbols onto the stack in reverse order (Y_2 then Y_1). A blank cell signals a syntax error.

2. Mechanics of LR(1) Parsers:

An LR(1) parser is a powerful bottom-up parsing framework. It reads tokens from **L**eft-to-right, builds a **R**ightmost derivation in reverse, and uses **1** explicit lookahead token attached directly to state items to

prevent parsing conflicts. It tracks operations using a symbol/state stack alongside two tables: **ACTION** and **GOTO**.

Based on the current state at the top of the stack and the next input token, the parser chooses between four main actions:

- **Shift (s_n):** Consumes the current input token, pushes it onto the stack, and switches the system to state `n`.
- **Reduce (r_k):** Triggered when a complete valid rule matches the top of the stack. It pops the matching symbols off the stack, checks the **GOTO** table using the unmasked state underneath, and pushes the new non-terminal symbol back onto the stack.
- **Accept:** Ends execution successfully when the full program is parsed and reduced back to the initial start symbol.
- **Error:** Triggered when a blank entry is encountered in the table, indicating invalid syntax.

QUESTION NO. 3 (Lexical Analysis Pipeline) [12 Marks]

A) What are the primary responsibilities of the lexical analyzer? [6 Marks]

The lexical analyzer acts as the entry point for raw code into a compiler. Its primary role is to read the source text character-by-character and convert it into a stream of structured tokens for the parser.

Its key responsibilities include:

1. **Tokenization:** Groups sequences of raw characters into text strings called *lexemes* and maps them to standard category tags called *tokens* (e.g., matching the characters `i-f` to the token type `KEYWORD_IF`).
2. **Stripping Extraneous Elements:** Automatically filters out whitespace, tabs, newlines, and comments, keeping the parser tables clean and focused.
3. **Symbol Table Insertion:** Identifies variable names, identifiers, and function names, adds them to the central Symbol Table, and returns their index values as token attributes.
4. **Source Error Location Mapping:** Tracks internal metadata like row numbers and column boundaries so the compiler can provide accurate locations for syntax errors.

B) How are regular expressions used to specify tokens, and how do finite automata implement these specifications? [6 Marks]

Token parsing relies on a simple two-step workflow: defining token patterns using text-based **Regular Expressions**, then running them efficiently using **Finite Automata** state machines.

1. Specifying Tokens with Regular Expressions:

Regular expressions use simple algebraic syntax rules like choices (`|`), sequence sequences, and repetition loops (`*`) to define valid patterns for different types of tokens:

```
Digit      →  0 | 1 | 2 | ... | 9
Letter     →  [a-zA-Z]
Identifier →  Letter ( Letter | Digit )*
```

2. Turning Patterns into State Diagrams (The Execution Flow):

To execute these regular expression rules programmatically, compilers convert them into state-based machines using a standard automated pipeline:

- **Thompson's Construction (RE → NFA):** Automatically builds a state graph containing empty step choices (`ε`) directly from the regular expression. This machine can have multiple path options for a single input character.
- **Subset Construction (NFA → DFA):** Simplifies the NFA graph into a deterministic machine (DFA) where each state has exactly one clear path for any given character. This creates a fast, predictable lookup framework that can run directly via code loops.

QUESTION NO. 4 (Syntax Structures & Parsers) [Solve Any Two • 12 Marks]

A) What is the difference between a parse tree and an abstract syntax tree? [6 Marks]

Parse Trees and Abstract Syntax Trees (ASTs) represent code structure at different levels of detail.

Feature	Parse Tree (Concrete Syntax)	Abstract Syntax Tree (AST)
Level of Detail	A complete, literal representation of every single grammar step used to parse the input string.	A clean, compact tree that captures the core meaning of the code while omitting unnecessary syntax clutter.
Node Contents	Internal nodes are structural non-terminals; leaves are raw text characters and punctuation marks.	Internal nodes represent operations (e.g., `+`, `=`); leaf nodes contain the actual variables or values.
Syntax Overhead	Includes explicit punctuation marks like parentheses, commas, colons, or semicolons.	Omits all punctuation and structural keywords; grouping rules are implied by the tree hierarchy.

TREE REPRESENTATION COMPARISON FOR STATEMENT: "X = A + B"

Parse Tree (Verbose, full grammar steps):

```
AssignStmt ├── id (x) ├── = └──
Expr ├── Expr ── id (a) ├── +
└── Expr ── id (b)
```

AST (Clean, direct execution layout):

```
= ├── id (x) └── + ├── id (a)
└── id (b)
```

B) What are the advantages and disadvantages of recursive descent parsers? [6 Marks]

A recursive descent parser is a top-down parsing technique where language grammar rules are written directly as a series of mutually recursive code functions.

Key Advantages:

- **Intuitive and Readable:** The code structure maps directly to the grammar rules, making it straightforward to write and maintain by hand.
- **No Table Generator Dependencies:** Runs natively without needing complex, automated table matrices, keeping the parser lightweight.
- **Simple Debugging:** Because execution follows a standard function call stack, you can easily step through lines with a standard debugger to spot errors.

Key Disadvantages:

- **Left-Recursion Hazards:** If the grammar has left-recursive rules (like $A \rightarrow A\alpha$), the parser will fall into infinite loops and crash from a stack overflow.

- **Performance Overhead:** Constant recursive function calls consume considerable processor time and memory overhead.
- **Backtracking Overhead:** If a parsing branch guess fails, resetting the input pointer and rolling back state can slow execution down significantly.

C) How does shift-reduce parsing work? [6 Marks]

Shift-reduce parsing is an efficient bottom-up strategy that uses an explicit storage stack alongside an incoming input buffer to reduce code strings back into the initial grammar start symbol.

The system evaluates the top items of the stack and lookahead tokens to perform four fundamental operations:

- **Shift:** Reads the next available token from the input buffer and pushes it onto the parsing stack.
- **Reduce:** Triggered when a sequence at the top of the stack matches the right-hand side of a valid rule ($A \rightarrow \beta$). This sequence is popped off and replaced by the non-terminal symbol A .
- **Accept:** Concludes parsing successfully if the input buffer is empty and the stack contains only the primary start symbol.
- **Error:** Invoked when no matching shift or reduction step can be found, triggering error recovery.

Common Parsing Conflicts: Ambiguous grammars can trigger two main types of conflicts:

- *Shift/Reduce Conflict:* The parser cannot decide whether to shift the next token or reduce the current items on the stack.
- *Reduce/Reduce Conflict:* The top items on the stack match multiple rules simultaneously, leaving the parser unable to determine which substitution rule to use.

QUESTION NO. 5 (Intermediate Code & Runtimes) [Solve Any Two • 12 Marks]

A) What are common forms of intermediate representations? [6 Marks]

Intermediate Representations (IR) provide an isolated, language-independent middle layer in the compiler that separates front-end parsing from back-end code generation. This allows optimizations to work across different source languages and target architectures.

1. Abstract Syntax Trees (AST): High-level graphical representations that capture the nesting structure of code without syntax clutter, making them ideal for tasks like type-checking.

2. Three-Address Code (TAC): A low-level linear code layout where each statement contains at most three addresses (two inputs and one output destination). Complex calculations are broken down into flat sequences using temporary variables:

```
Original statement: x = (a + b) * (c - d)
Linearized TAC format:
    t1 = a + b
    t2 = c - d
    t3 = t1 * t2
    x  = t3
```

TAC can be stored in memory using three formats: **Quadruples** (explicit records with fields: op, arg1, arg2, result), **Triples** (omits the result column; steps refer directly to preceding array indices), or **Indirect Triples** (decouples instructions using pointers, making it easy to reorder lines during optimization).

3. Control Flow Graphs (CFG): A directed graph mapping out all execution paths in a program. Nodes represent **Basic Blocks** (straight-line blocks of code with a single entry and exit point), while edges track control changes like loops or conditional branches.

B) How are function calls and returns managed in the generated code? [6 Marks]

Function execution relies on a dedicated memory block called an **Activation Record (Stack Frame)**, managed on the system call stack.

Stack Frame Component	Purpose
Parameters	Stores arguments passed by the caller into the function.
Return Value Slot	Dedicated memory space where the function writes its output value before exiting.
Control Link (Dynamic Link)	Points back to the base address of the caller's stack frame, allowing it to be restored on exit.
Saved Machine Status	Backs up critical CPU registers and the Return Address pointer before branching.
Local Variables	Allocates internal memory space for variables declared locally inside the function.

The Execution Cycle: The caller evaluates arguments, saves its registers, and runs a branch instruction (like `CALL`) to push the return address and jump to the function. The called function adjusts

the stack pointer to reserve space for its local variables, executes its code, writes its return value, restores the saved CPU registers, and runs a return instruction (like RET) to jump back to the caller.

C) How do these optimizations improve the efficiency of the generated code? [6 Marks]

Optimizations applied to Intermediate Representations directly improve program efficiency by saving processor time and minimizing hardware resource use:

1. **Speed Optimization:** Techniques like *Constant Folding* calculate constant values during compilation (e.g., replacing `2 * 3.14` with `6.28`), eliminating math operations at runtime. *Loop-Invariant Code Motion* moves calculations that yield the same result outside of loops, preventing redundant operations from executing millions of times inside hot loops.
2. **Memory Reductions:** *Dead Code Elimination* detects and removes variables or instructions that are never used or reached, reducing the final binary size and improving CPU cache performance.
3. **Strength Reduction:** Replaces expensive CPU operations with cheaper alternatives (e.g., converting a slow multiplication like `x * 8` into a fast bitwise left shift: `x << 3`).

QUESTION NO. 6 (Advanced Optimizations) [Solve Any Two • 12 Marks]

A) What are the differences between high-level and low-level optimizations? [6 Marks]

Optimizations operate at different stages of compilation, falling into either high-level or low-level categories depending on how close they are to the source language versus the target hardware.

Feature	High-Level Optimizations	Low-Level Optimizations
Target Phase	Applied early on abstract representations like Abstract Syntax Trees (ASTs) or high-level IRs.	Applied late in the pipeline on low-level linear representations, assembly, or machine code instructions.
Hardware Dependency	Completely hardware-independent; changes work universally across any CPU type.	Highly hardware-dependent; tailored to specific CPU registers, pipelines, and instruction sets.
Primary Focus	Optimizes program logic, block structures, and execution flow.	Optimizes hardware performance, memory addresses, and register assignments.
Common Examples	Function Inlining, Loop Unrolling, Dead Code Elimination.	Register Allocation (Graph Coloring), Peephole Optimizations, Instruction Scheduling.

B) What is data flow analysis and how is it used in optimization? [6 Marks]

Data-flow analysis is a mathematical framework used to collect information about how data moves through a program's Control Flow Graph (CFG). It calculates properties at the entry and exit points of Basic Blocks by solving equations that combine local block adjustments with path intersection sets:

$$IN[B] = \bigcup_{P \in Pred(B)} OUT[P] \quad \bullet \quad OUT[B] = gen_B \cup (IN[B] - kill_B)$$

Here, gen_B tracking properties created inside block B, while $kill_B$ accounts for properties invalidated or overwritten within that same block.

Key Practical Applications:

- **Reaching Definitions:** Tracks where each variable value was created and where it flows without being overwritten, enabling safe *Constant Propagation*.
- **Live Variable Analysis:** Determines if a variable's current value will be read downstream before it gets overwritten. If it is found to be "dead," the compiler can reuse its register, reducing memory spills.
- **Available Expressions:** Checks if a math calculation has already been evaluated along all incoming paths without its values changing, allowing for *Common Subexpression Elimination (CSE)*.

C) What are the primary goals of code optimization? [6 Marks]

The primary objective of code optimization is to transform a program to improve execution efficiency while following strict correctness guarantees.

Its primary goals include:

- **Preserving Semantic Correctness (The Golden Rule):** Transformations must never alter the observable behavior or output logic of the original code. The optimized code must produce identical results for all possible inputs.
- **Maximizing Execution Speed:** Aims to minimize the total number of CPU clock cycles needed for execution, reducing program latency.
- **Minimizing Resource Footprints:** Minimizes RAM usage and reduces final binary size, which is critical for resource-constrained or embedded environments.
- **Reducing Power Consumption:** Eliminating redundant steps helps minimize energy use, which is essential for mobile and battery-powered systems.